Connect to a Server with SSL: How to Ignore Verification

Explore how to handle server connections with SSL in Python by learning to ignore certificate verification when working with self-signed certificates. This lesson teaches you to use the requests library's ssl_verify option to manage secure connections and handle exceptions effectively.

How to ignore verification

A certificate signed by a Certificate Authority (CA) forms the basis of HTTPS-enabled websites and services. The CA itself is a certificate (cert) that is used to sign other certs. However, if the CA cert used by the service isn't present in the client's system, then there are two approaches:

1. Ignore the cert verification process and proceed to fetch data from the endpoint.
2. Load the CA cert manually, pass it on to the verification process, and then proceed to fetch data from the endpoint.

The most common reason for a CA cert to not be present in the system is that the server cert is self-signed, i.e., the owner of the service has issued both the server cert and the CA cert. From a security standpoint, approach 1 should be used only until the CA cert is obtained from the owner of the service. Once it's obtained, approach 2 should replace approach 1.

Ask

In this lesson, we'll look at how to ignore the verification of certificates. We'll use the `requests` library that we decided upon in the last lesson. The service that

we'll use for checking our REST client code will be https://self-signed.badssl.com, as it provides a self-signed certificate for testing purposes.

> **Note:** We'll be building upon the example created in the last lesson to create two REST clients, one with SSL verification enabled and one without, to demonstrate both responses.

The `requests` library accepts a set of options to configure the underlying connection. One of them relates to the verification of SSL certs.

Without further ado, let's start.

Steps

1. Open `client.py` created in the last lesson.
2. Add a parameter named `ssl_verify` to the constructor, i.e., `__init__` method. Set its default to `True`.

```
def __init__(self, url: str, ssl_verify=True):
```

3. Assign the `ssl_verify` parameter passed to the constructor to an instance-level variable with the same name, i.e., `ssl_verify`.

```
def __init__(self, url: str, ssl_verify=True):
  self.url = url
  self.ssl_verify = ssl_verify
```

4. Update the `get` method so that it passes the value of the `ssl_verify` instance-level variable to the `get` method of the `requests` library.

```
def get(self) -> str:
  return requests.get(self.url,verify=self.ssl_verify).content
```

Ask

5. Remove the existing code from the main block that we had created in the last lesson.
6. Create the first instance of `Client` named `client_verify` in the main block by passing the URI of the bad SSL service.

```
client_verify = Client('https://self-signed.badssl.com')
```

7. Add a `try/except` block that catches all types of exceptions. In the `except` block, add a `print` statement that prints out the exception that has been caught.

```
try:

except Exception as e:
  print("Client with Verification Enabled")
  print("Caught exception", e)
```

8. Call the `get` method of `client_verify` within the `try` block and print the result.

```
print(client_verify.get())
```

9. Create the second instance of `Client` named `client_no_verify` after the `try/except` block by passing the URI of the bad SSL service and setting the `ssl_verify` parameter to false.

```
client_no_verify = Client('https://self-signed.badssl.com', ssl_verify=False
```

10. Call the `get` method of the `client_no_verify` and print the result.

```
print("Client with Verification Disabled")
print(client_verify.get())
```

11. Run the script. Two lines will be printed out. The first line corresponds exception when the method of `client_verify` is called. The second line corresponds to the response from the service.

Ask

Show client.py output

How it works

The core of the logic is in the `get` method of the `Client`, specifically the `ssl_verify` option that is being passed to the `get` method of requests. The `ssl_verify` option tells requests whether or not to verify the cert of the service.

The following diagram depicts the request-response cycle between the client and the server when client verification is enabled:

Client with verification enabled

The following diagram depicts the request-response cycle between the client and the server when client verification is disabled:

Ask

Client with verification disabled

In the constructor of the `Client`, the default value of the `ssl_verify` is se

Ask

`True`. Hence, when the `get` method of the `client_verify` instance calls the `get`

method, `requests` verifies the SSL cert and throws an exception related to cert verification.

Whereas, when the `client_no_verify` is instantiated, we pass `False` as the value of the `ssl_verify` parameter. This, in turn, tells requests not to verify the cert of the service. Hence, when the `get` method of `client_no_verify` is called, the response is printed.

Try it yourself

- Follow the steps above to complete the code for this lesson.
- Click “Run” to execute the code and display the result.

client.py Python 3.8

```
## The code goes here
#
#
```

Run

The client.py code with and without verification

Ask

Solution

If stuck, refer to the final code below for help.

Please note that the file below is read-only.

client.py Python 3.8

```
from ssl import SSLError, SSLCertVerificationError

import requests


class Client(object):
    def __init__(self, url: str, ssl_verify=True):
        self.url = url
        self.ssl_verify = ssl_verify
```

Ask